

PRACTICAL FILE

**COMPUTER GRAPHIC AND
ANIMATIONS**

LAB

(BCA-E551)



Bachelors of Computer Applications

Department of computing Sciences

Uttaranchal School of Computing Sciences

Session (2023-2024)

Submitted To:

Ms.Rubby Faizan

Associate Professor

USCS, Uttaranchal University

Submitted By:

Shivang Chhetri

BCA-5th Sem-(B)

Enroll No-UU2109000102

INDEX

S.No.	Date	Practicals	Page No.	Signature
1.		Design a Cube using OpenGL Programming.		
2.		Implement a program to create chess board using DDA line algorithm		
3.		Implement a Bresenham's line drawing algorithm with all values.		
4.		Design a program to implement mid-point circle generating algorithm.		
5.		WAP to implement Liang-Basky Line Clipping Algorithm with Window to viewport Mapping		
6.		WAP to implement a program to create polygon and fill the color by using boundary fill algorithm		
7.		Design and implement a program to create a program to create a polygon and fill the yellow color by using scan-line area filling algorithm.		
8.		Design and implement a color cube and spin it using Opengl Transformation matrices.		
9.		Design and implement a program to create a hut and rotate it about a given fixed point using OpenGL function.		
10.		Design and implement a program to create a triangle and scale it along X axis, Y-axis and XY plane.		

PRACTICAL-01

Q1:- Draw a cube using OpenGL

```
#include <stdlib.h>
#include<stdio.h>
#include <GL/gl.h>
#include <GL/glut.h>

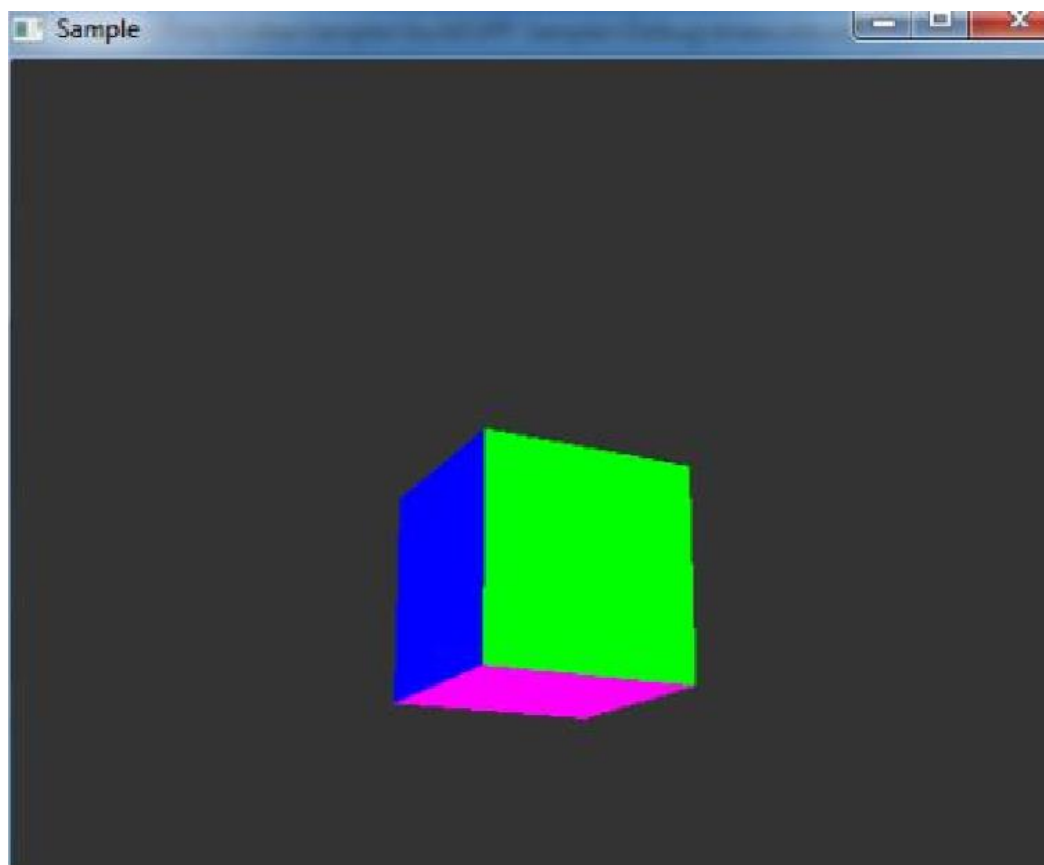
GLfloat angle = 0.0; // Initial rotation angle
void display() {
    glClear (GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity ();
    glTranslatef (0.0, 0.0,-5.0); // Move the cube away from the camera
    // glRotatef(angle, 1.0, 1.0, 0.0); // Rotate the cube
    // Draw the cube
    glutWireCube(2.0);
    glutSwapBuffers();
}

Void reshape (int width, int height)
{
    glViewport (0, 0, width, height);
    glMatrixMode (GL_PROJECTION);
    glLoadIdentity ();
    gluPerspective (45.0, (GLfloat)width / (GLfloat)height, 0.1, 100.0);
    glMatrixMode (GL_MODELVIEW);
}

Void idle ()
{
    angle += 0.01; // Increment rotation angle if (angle > 360.0)
    angle -= 360.0;
    glutPostRedisplay (); // Request a redraw
}

int main(int argc, char** argv) {
    glutInit (&argc, argv);
    glutInitDisplayMode (GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutCreateWindow ("OpenGL Cube Example");
    glEnable (GL_DEPTH_TEST);
    glClearColor (0.0, 0.0, 0.0, 1.0);
    glutDisplayFunc (display);
    glutReshapeFunc (reshape);
    glutIdleFunc (idle);

    glutMainLoop ();
    return 0;
}
```

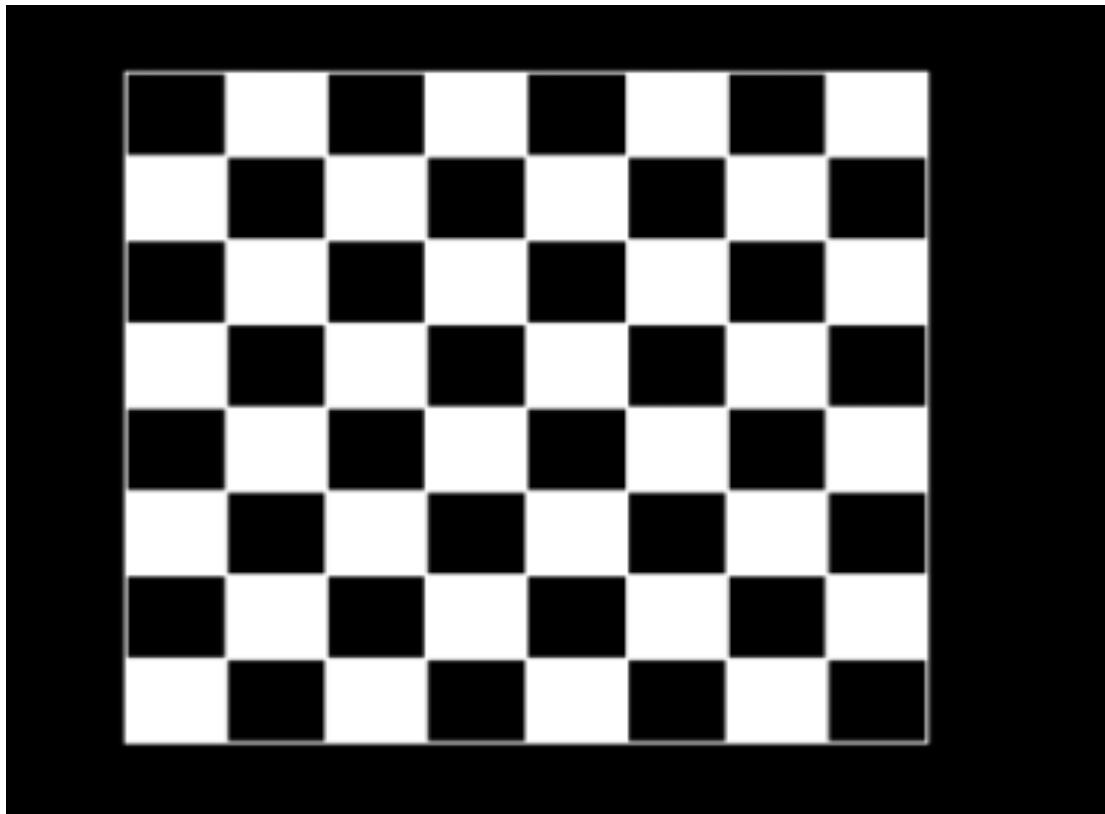


PRACTICAL-02

Q2: Design and develop and Implement a program to create a chess board DDA line algorithm:-

```
#include <graphics.h>
#include<conio.h>
#include<stdio.h>
void drawLine (int x1, int y1, int x2, int y2)
{
    int dx = x2 - x1;
    int dy = y2 - y1;
    int steps = abs (dx) > abs (dy) ? abs (dx) : abs (dy);
    float xIncrement = (float)dx / (float)steps;
    float yIncrement = (float)dy / (float)steps;
    float x = x1, y = y1;
    for (int i = 0; i <= steps; i++)
    {
        putpixel (round (x), round (y),
            WHITE); // You can change the color as needed x += xIncrement;
        y += yIncrement;
    }
}
void drawChessBoard ()
{
    int gd = DETECT, gm;
    initgraph (&gd, &gm, "");
    int squareSize = 50;
    int startX = 50, startY = 50;
    for (int i = 0; i < 8; i++)
    {
        for (int j = 0; j < 8; j++)
        {
            int x1 = startX + j * squareSize;
            int y1 = startY + i * squareSize;
            int x2 = x1 + squareSize;
            int y2 = y1;
            drawLine (x1, y1, x2, y2);
            x2 = x1;
            y2 = y1 + squareSize;
            drawLine (x1, y1, x2, y2);
        }
    }
    delay (5000);
    closegraph ();
}
```

```
int main ()  
{  
    drawChessBoard ();  
    return 0;  
}
```



PRACTICAL-03

Q3: Design a program to implement Bresenham's line drawing algorithm with all values of slopes:-

```
#include <GL/glut.h>
#include<stdio.h>
int x1, y1, x2, y2;
void draw_pixel (int x, int y)
{
    glColor3f(1.0, 0.0, 0.0);
    glBegin (GL_POINTS);
    glVertex2i (x, y);
    glEnd ();
}
void bresenham (int x1, int y1, int x2, int y2)
{
    int dx = x2 - x1; // x difference
    int dy = y2 - y1; // y difference int m = dy/dx; // slope
    if (m < 1)
    {
        int decision = 2 * dy - dx;
        int x = x1; // initial x
        int y = y1; // initial y
        if (dx < 0) // decide the first point and second point { x = x2; // making
            // second point as first point
            y = y2;
            x2 = x1;
        }
        draw_pixel (x, y); // plot a point
        while (x < x2) // from 1st point to 2nd point { if (decision >= 0) {
            x = x + 1;
```

```

    y = y + 1;
    decision = decision + 2 * dy - 2 * dx;
    ;
}
else
{
    x = x + 1;
    y = y;
    decision = decision + 2 * dy - 2 * dx;
}
draw_pixel (x, y);
}
}
else if (m > 1)
{
    int decision = 2 * dx - dy;
    int x = x1; // initial x

    int y = y1; // initial y if (dy < 0) {
    x = x2;
    y = y2;
    y2 = y1;
    }
    draw_pixel (x, y);
    while (y < y2)
    {
        if (decision >= 0)
        {
            x = x + 1;
            y = y + 1;

```



```

        decision = decision + 2 * dx - 2 * dy;
    }
else
{
    y = y + 1;
    x = x;
    decision = decision + 2 * dx - 2 * dy;
}
draw_pixel (x, y);
}
}
else if (m == 1)
{
    int x = x1;
    int y = y1;
    draw_pixel (x, y);
    while (x < x2)
    {
        x = x + 1;
        y = y + 1;
        draw_pixel (x, y);
    }
}
}
void
init ()
{
    glClearColor (1, 1, 1, 1);
    gluOrtho2D (
        0.0, 500.0, 0.0,

```

```

        500.0); // left ->0, right ->500, bottom ->0, top ->500} void display(){
glClear (GL_COLOR_BUFFER_BIT);
bresenham (x1, y1, x2, y2);
glFlush ();
}
int main (int argc, char **argv)
{
printf ("Enter Start Points (x1,y1)\n");
scanf ("%d %d", &x1, &y1); // 1st point from user printf( "Enter End Points (x2,y2)\n");
scanf ("%d %d", &x2, &y2); // 2nd point from user glutInit(&argc, argv); //
        // initialize graphics system

glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
glutInitWindowSize ( 500, 500); // 500 by 500
window size glutInitWindowPosition(220, 200); //
        // where do you wanna see your window
        // glutCreateWindow("Bresenham's Line Drawing"); // the title
        // of your window init(); // initialize the canvas
glutDisplayFunc ( display); // call display function glutMainLoop(); // run forever
}

```



```
NeuTroN DOS-C++ 0.77, Cpu speed: max 100% cycles, Frameskip 0, Program:  
Enter co-ordinates of first point: 100  
100  
Enter co-ordinates of second point: 200  
200
```



PRACTICAL-04

Q4: Design a Program to implement mid-point circle generation algorithm.

```
#include <stdio.h>
#include <iostream>
#include <GL/glut.h>
using namespace std;
int pointX1, pointY1, r;
void plot(int x, int y){
    glBegin(GL_POINTS);
    glVertex2i(x + pointX1, y + pointY1);
    glEnd();
}
void myInit(void) {
    glClearColor(1.0, 1.0, 1.0, 0.0);
    glColor3f(0.0f, 0.0f, 0.0f);
    glPointSize(4.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0.0, 640.0, 0.0, 480.0);
}
void circle() {
    int x = 0;
    int y = r;
    float decision = 1 - r;
    plot(x, y);
    while (x > y) {
        if (decision < 0) {
            x++;
            decision += 2 * x + 1 + 1;
```

```

    } else {
        y--;
        x++;
        decision += 2 * (x - y) + 1;
    }
    plot(x, y);
    plot(x, -y);
    plot(-x, y);
    plot(-x, -y);
    plot(y, x);

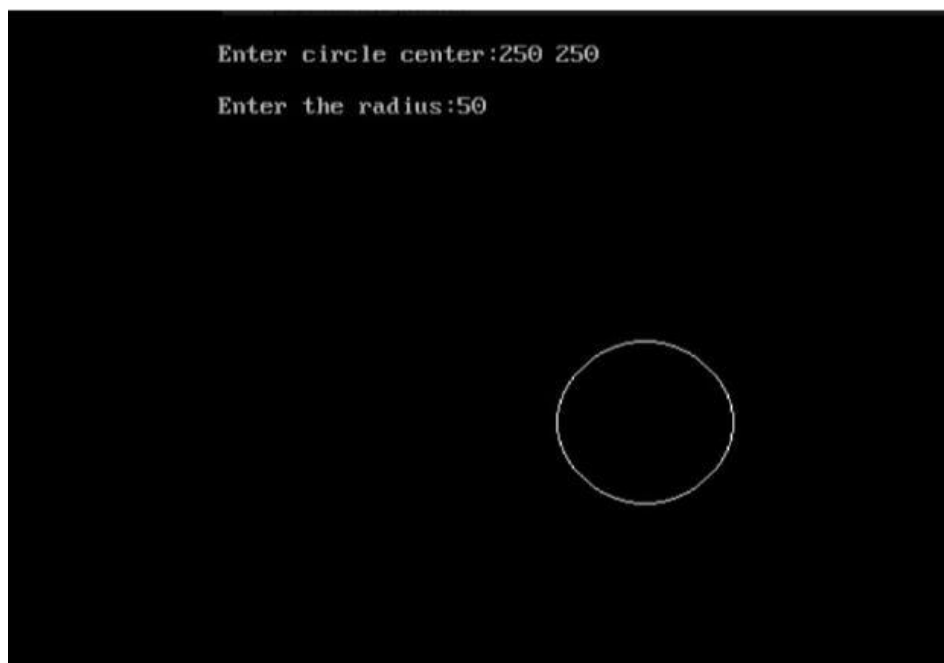
    plot(-y, x);
    plot(y, -x);
    plot(-y, -x);
}
}

void myDisplay(void) {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.0, 0.0, 0.0);
    glPointSize(1.0);
    circle();
    glFlush();
}

void main(int argc, char** argv) {
    cout << "Enter the coordinates of the center:\n\n" << endl;
    cout << "X-coordinate : ";
    cin >> pointX1;
    cout << "\nY-coordinate : ";
    cin >> pointY1;
    cout << "\nEnter radius : ";

```

```
cin >> r;  
glutInit(&argc, argv);  
glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);  
glutInitWindowSize(640, 480);  
glutInitWindowPosition(100, 150);  
glutCreateWindow("Line Drawing Alogrithms");  
glutDisplayFunc(myDisplay);  
myInit();  
glutMainLoop();  
}
```



PRACTICAL-05

Q5: Design a Program to implement Liang-Barsky Line Clipping Algorithm with Window to viewport Mapping.

```
#include <stdio.h>

#include <graphics.h>

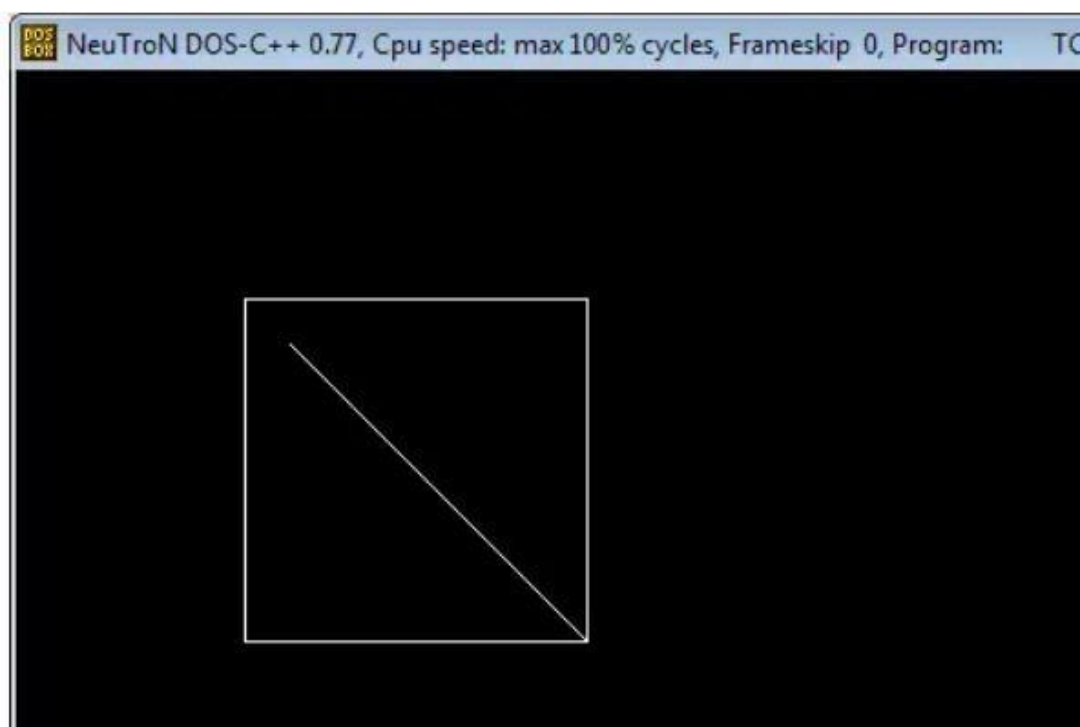
// Function to implement Liang-Barsky Line Clipping Algorithm
void liangBarsky(int x1, int y1, int x2, int y2, int xmin, int ymin, int xmax,
                int ymax) {
    int p[4], q[4], i;
    float t1 = 0, t2 = 1;
    int dx = x2 - x1;
    int dy = y2 - y1;
    p[0] = -dx;
    p[1] = dx;
    p[2] = -dy;
    p[3] = dy;
    q[0] = x1 - xmin;
    q[1] = xmax - x1;
    q[2] = y1 - ymin;
    q[3] = ymax - y1;
    for (i = 0; i < 4; i++) {
        if (p[i] == 0) {
            if (q[i] < 0) {
                printf("Line is parallel to clipping boundary and outside\n");
                return;
            }
        } else {
            float t = (float)q[i] / p[i];
            if (p[i] < 0 && t > t1)
```

```

        t1 = t;
    else if (p[i] > 0 && t < t2)
        t2 = t;
    }
}
if (t1 < t2) {
    int x1_clip = x1 + t1 * dx;
    int y1_clip = y1 + t1 * dy;
    int x2_clip = x1 + t2 * dx;
    int y2_clip = y1 + t2 * dy;
    int vx1 = (int)(x1_clip * 100);

    int vy1 = (int)(y1_clip * 100);
    int vx2 = (int)(x2_clip * 100);
    int vy2 = (int)(y2_clip * 100);
    line(vx1, vy1, vx2, vy2);
}
}
int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");
    int xmin = 50, ymin = 50, xmax = 200, ymax = 200;
    rectangle(xmin * 100, ymin * 100, xmax * 100, ymax * 100);
    line(x1 * 100, y1 * 100, x2 * 100, y2 * 100);
    liangBarsky(x1, y1, x2, y2, xmin, ymin, xmax, ymax);
    getch();
    closegraph();
    return 0;
}

```

PRACTICAL-06

Q6: Design and implement a program to create a polygon and fill the color by using boundary fill algorithm.

***Boundary Fill Algorithm * /**

```
#include<stdio.h>
#include<conio.h>
#include<graphics.h>

void boundaryFill4(int x, int y, int fill_color, int boundary_color) {
if (getpixel(x, y) != boundary_color && getpixel(x, y) != fill_color) {
    putpixel(x, y, fill_color);
    boundaryFill4(x + 1, y, fill_color, boundary_color);
    boundaryFill4(x, y + 1, fill_color, boundary_color);
    boundaryFill4(x - 1, y, fill_color, boundary_color);
    boundaryFill4(x, y - 1, fill_color, boundary_color);
}
}

// driver code int main()
{
    // gm is Graphics mode which is
    // a computer display mode that
    // generates image using pixels.
    // DETECT is a macro defined in
    // "graphics.h" header file int gd = DETECT, gm; int x , y , radius;

    // initgraph initializes the
    // graphics system by loading a
    // graphics driver from disk
    printf("\n\n Enter x,y coordinates and radius of circle: ");
```

```
scanf("%d%d%d", &x, &y, &radius);  
initgraph(&gd, &gm, "C:\\TURBOC3\\BGI");  
  
// circle fuction circle(x, y, radius);  
  
// Function calling boundaryFill4(x, y, 6, 15);  
  
delay(10000);  
getch();  
// closegraph function closes the  
// graphics mode and deallocates  
// all memory allocated by  
// graphics system . closegraph();  
  
return 0;  
}
```



Enter the center point :100 200
Enter the value for Rx and Ry :25 45

PRACTICAL-07

Q7: Design and implement a program to create a polygon and fill the yellow color by using scan-line area filling algorithm.

```
#include <stdio.h>

#include <graphics.h>

void drawPolygon(int num, int polyPoints[][2]) {
    int i;
    for (i = 0; i < num - 1; i++) {
        line(polyPoints[i][0], polyPoints[i][1], polyPoints[i + 1][0],
            polyPoints[i + 1][1]);
    }
    // Closing the polygon
    line(polyPoints[i][0], polyPoints[i][1], polyPoints[0][0], polyPoints[0][1]);
}

void scanLineFill(int y, int x1, int x2) {
    int x;
    for (x = x1; x <= x2; x++) {
        putpixel(x, y, YELLOW);
    }
}

void fillPolygon(int num, int polyPoints[][2]) {
    int minY = getmaxy(); // Set to the maximum y-coordinate initially
    int maxY = 0; // Set to the minimum y-coordinate initially

    // Find the minimum and maximum y-coordinates of the polygon for (int i = 0; i
    // < num; i++) {
    if (polyPoints[i][1] < minY) {
        minY = polyPoints[i][1];
    }
    if (polyPoints[i][1] > maxY) {
```

```

        maxY = polyPoints[i][1];
    }
}

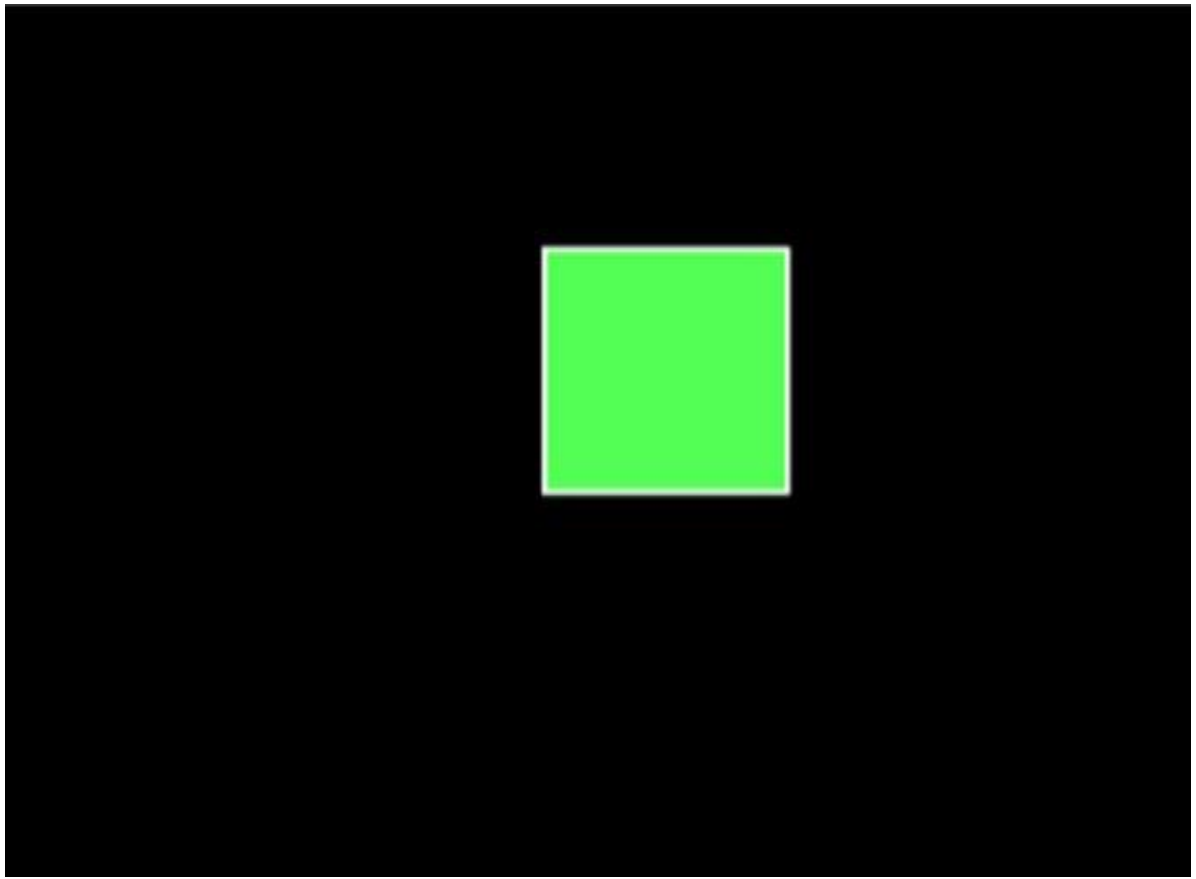
// Scan-line filling
for (int y = minY; y <= maxY; y++) {
    int intersectX[20], k = 0;
    for (int i = 0; i < num; i++) {
        int j = (i + 1) % num; // The next vertex
        int y1 = polyPoints[i][1];
        int y2 = polyPoints[j][1];
        if ((y1 <= y && y2 > y) || (y2 <= y && y1 > y)) {
            intersectX[k++] =
                polyPoints[i][0] +
                (y - y1) * (polyPoints[j][0] - polyPoints[i][0]) / (y2 - y1);
        }
    }

    // Sort the intersection points
    for (int i = 0; i < k - 1; i++) {
        for (int j = 0; j < k - i - 1; j++) {
            if (intersectX[j] > intersectX[j + 1]) {
                int temp = intersectX[j];
                intersectX[j] = intersectX[j + 1];
                intersectX[j + 1] = temp;
            }
        }
    }

    // Fill the scan-line with color
    for (int i = 0; i < k; i += 2) {
        scanLineFill(y, intersectX[i], intersectX[i + 1]);
    }
}
}
}

```

```
int main() {  
    int gd = DETECT, gm;  
    initgraph(&gd, &gm, "");  
    int num = 4; // Number of vertices of the polygon  
    int polyPoints[4][2] = {{100, 200}, {200, 100}, {300, 200}, {200, 300}};  
    drawPolygon(num, polyPoints);  
    fillPolygon(num, polyPoints);  
    delay(5000);  
    closegraph();  
    return 0;  
}
```



PRACTICAL-08

Q8: Design and implement a color cube and spin it using OpenGL transformation matrices.

```
#include <stdlib.h>
#include<stdio.h>
#include <GL/gl.h>
#include <GL/glut.h> GLfloat T = 0;

void Spin() {
    T = T + 0.5;
    if (T > 360) T = 0;
    glutPostRedisplay();
}

void myInit(void) {
    glClearColor(0, 0, 0, 1);
    glColor3f(1, 0, 0);
    glEnable(GL_DEPTH_TEST);
}

void Face(GLfloat A[], GLfloat B[], GLfloat C[], GLfloat D[]) {
    glBegin(GL_POLYGON);
    glVertex3fv(A);
    glVertex3fv(B);
    glVertex3fv(C);
    glVertex3fv(D);
    glEnd();
}

void Cube(GLfloat V0[], GLfloat V1[], GLfloat V2[], GLfloat V3[], GLfloat V4[],
          GLfloat V5[], GLfloat V6[], GLfloat V7[]) {
    glColor3f(1, 0, 0);
    Face(V0, V1, V2, V3);
    glColor3f(0, 1, 0);
```

```

Face(V4, V5, V6, V7);
glColor3f(0, 0, 1);
Face(V0, V3, V7, V4);
glColor3f(1, 0, 1);
Face(V1, V6, V2, V5);
glColor3f(1, 1, 0);
Face(V0, V1, V5, V4);

glColor3f(0, 1, 1);
Face(V3, V2, V7, V6);
}

void Draw() {
    GLfloat V[8][3] = {{-0.5, 0.5, 0.5}, {0.5, 0.5, 0.5}, {0.5, -0.5, 0.5},
                       {-0.5, -0.5, 0.5}, {-0.5, 0.5, -0.5}, {0.5, 0.5, -0.5},
                       {0.5, -0.5, -0.5}, {-0.5, -0.5, -0.5}};

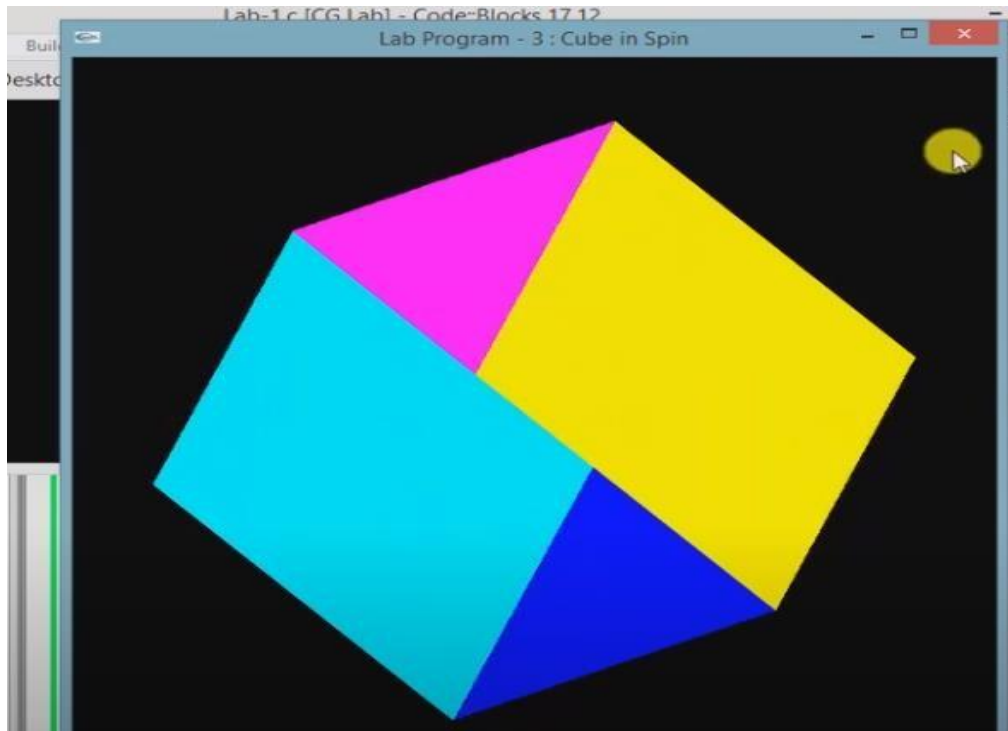
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    glRotatef(T, 0, 1, 0);
    Cube(V[0], V[1], V[2], V[3], V[4], V[5], V[6], V[7]);
    glutSwapBuffers();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
    glutInitWindowPosition(250, 50);
    glutInitDisplayMode(GLUT_RGB | GLUT_DOUBLE | GLUT_DEPTH);
    glutCreateWindow("cube");
    myInit();
    glutDisplayFunc(Draw);
    glutIdleFunc(Spin);
}

```



```
glutMainLoop();  
return 0;  
}
```



PRACTICAL-09

Q9: Design and implement a Program to create a hut and rotate it about a given fixed point

using OpenGL functions.

```
#include <GL/glut.h>
```

```
#include <math.h>
```

```
GLfloat hutVertices[][2] = {
```

```
{50, 50}, {250, 50}, {250, 250}, {50, 250}, {150, 350}};
```

```
GLfloat fixedPoint[] = {150, 250};
```

```
GLfloat angle = 0.0;
```

```
void drawHut() {
```

```
    glBegin(GL_LINE_LOOP);
```

```
    for (int i = 0; i < 5; i++) {
```

```
        glVertex2fv(hutVertices[i]);
```

```
    }
```

```
    glEnd();
```

```
    glBegin(GL_LINES);
```

```
    glVertex2fv(hutVertices[0]);
```

```
    glVertex2fv(hutVertices[2]);
```

```
    glEnd();
```

```
// Door glBegin(GL_LINE_LOOP); glVertex2f(120, 50);
```

```
    glVertex2f(180, 50);
```

```
    glVertex2f(180, 150);
```

```
    glVertex2f(120, 150);
```

```
    glEnd();
```

```
}
```

```
void rotateHut() {
```

```
    glTranslatef(fixedPoint[0], fixedPoint[1], 0.0);
```

```
    glRotatef(angle, 0.0, 0.0, 1.0);
```

```
    glTranslatef(-fixedPoint[0], -fixedPoint[1], 0.0);
```

```

}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0, 1.0, 1.0); // Set color to white drawHut();
    glColor3f(0.0, 1.0, 0.0); // Set color to green rotateHut();
    drawHut();

    glutSwapBuffers();
}

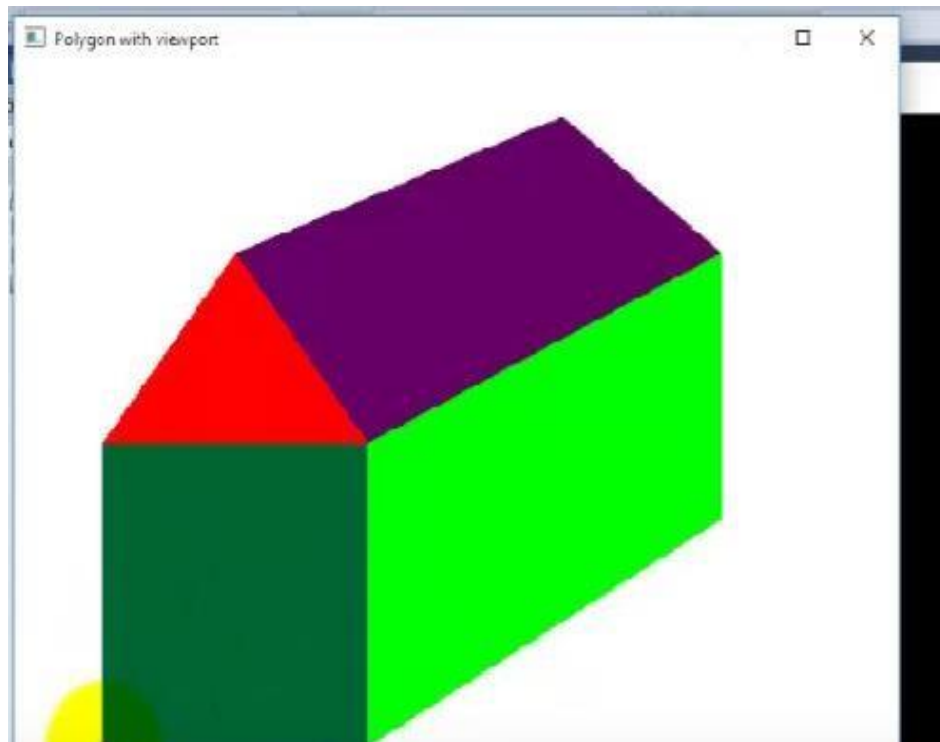
void reshape(int width, int height) {
    glViewport(0, 0, width, height);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0.0, width, 0.0, height);
    glMatrixMode(GL_MODELVIEW);
}

void idle() {
    angle += 1.0; // Increment rotation angle    if (angle > 360.0)        angle -=
        // 360.0; glutPostRedisplay(); // Request a redraw}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(400, 400);
    glutCreateWindow("Rotating Hut");
    glClearColor(0.0, 0.0, 0.0, 0.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0.0, 400.0, 0.0, 400.0);
    glutDisplayFunc(display);
    glutReshapeFunc(reshape);
}

```

```
glutIdleFunc(idle);  
glutMainLoop();  
return 0;  
}
```



PRACTICAL-10

Q10: Design and implement a program in C to create a triangle and scale it along Xaxis , Y axis and XY plane.

```
#include <graphics.h>
#include<conio.h>
#include<stdio.h>
void main() {
    int gd = DETECT, gm;
    int x1, x2, y1, y2, x3, y3, dx, dy;
    initgraph(&gd, &gm, "C:\\TURBOC3\\BGI");
    printf("Scaling of triangle\n");
    printf("Enter coordinates of a: ");
    scanf("%d%d", &x1, &y1);
    printf("Enter coordinates of b: ");
    scanf("%d%d", &x2, &y2);
    printf("Enter coordinates of c: ");
    scanf("%d%d", &x3, &y3);
    line(x1, y1, x2, y2);
    line(x2, y2, x3, y3);
    line(x3, y3, x1, y1);
    printf("Enter the scaling factor for x: ");
    scanf("%d", &dx);
    printf("Enter the scaling factor for y: ");
    scanf("%d", &dy);
    line(x1 * dx, y1 * dy, x2 * dx, y2 * dy);
    line(x2 * dx, y2 * dy, x3 * dx, y3 * dy);
    line(x3 * dx, y3 * dy, x1 * dx, y1 * dy);
    getch();
    closegraph();
}
```

}

```
Scaling of triangle
Enter coordinates of a: 150 200
Enter coordinates of b: 150 150
Enter coordinates of c: 100 120
Enter the scaling factor for x:
```

